Hybrid træning af en 2D racing-agent: Genetiske algoritmer og reinforcement learning

Sofia Emilia Abrahamsen, Harun Hursid, Betul Kazanasmazoglu Turkoglu

January 2, 2026

Abstract

We compare three approaches for learning control in a 2D Pygame racing environment. GA-only (NEAT), RL-only (DDQN), and a hybrid GA+RL model. The agent uses distance sensors to the track boundaries and selects discrete driving actions. All models are trained on Track 1 and evaluated on an unseen Track 2. During training, RL-only improves over episodes and GA-only improves over generations, while the hybrid combines NEAT's steering hint with DDQN's action selection. On Track 2, performance is measured as steps survived before crashing at max 1000. Results show that RL-only and the hybrid both reach the 1000-step, cap no crash within the test limit, whereas GA-only crashes quickly, indicating weaker generalization for GA-only in this setup.

1 Introduktion

I dette projekt udvikler vi et 2D racerspil i Pygame, hvor en agent skal lære at køre på en racerbane uden at ramme banens grænser. Vi undersøger to læringstilgange, genetiske algoritmer (GA) og reinforcement learning (RL). GA forbedrer adfærd gennem evolution over generationer via selektion og mutation, mens RL lærer gennem trial-and-error ved hjælp af belønning og straf.[4]

Formålet er at sammenligne tre modeller i vores miljø, altså en GA-only model, en RL-only model og en hybridmodel, der kombinerer GA med NEAT og RL med DDQN. Vi træner på Track 1 som er træningsbanen og tester derefter de færdigtrænede agenter på Track 2 (uset/test bane) for at vurdere generalisering.

Problemformulering. *Hvordan kan en kombination af genetiske algoritmer(GA) og reinforcement learning(RL) forbedre en agents evne til at navigere en 2D-køre bane uden kollisioner, sammenlignet med brugen af én metode alene, og i hvor høj grad kan en sådan hybrid-model generalisere til at køre på en ny, uset bane?*

Underspørgsmål.

1. *Hvordan udvikler agentens performance og læringsmønstre sig på tværs af GA-only, RL-only og hybridmodellen?*
2. *Hvordan performer den færdigtrænede hybrid-model på en ny, uset bane i forhold til GA-only og RL-only?*

2 Metode

Vi har implementeret et 2D racing-miljø i Pygame. Agenten får input fra sensorer, vælger handlinger og modtager belønning/straf baseret på fremskridt og kollisioner. Træning foregår på Track 1 træningsbanen og evaluering på Track 2 testbanen uden videre læring.

I GA-delen bruges track-billedets græsfarve som "væg", mens RL- og hybridmiljøet repræsenterer banekanterne som linjestykker. Begge miljøer understøtter resize gennem skalering, så alle kan gøre brug af det.

På testbanen sammenligner vi GA, RL og Hybrid på en uset bane uden goals/gates. Alle tre biler starter ens og modtager samme sensorinput (8 rays + speed); GA-input re-mappes så det matcher GA-træningen, RL vælger action via argmax(Q), og Hybrid vælger action ud fra RL's Q-values med et NEAT-bias. Run stopper ved kollision/out-of-bounds eller max steps, og vi måler bl.a. overlevelse (steps) og penalties, som gives når bilen kolliderer, kører ud af banen, kommer for tæt på vægge, eller holder for lav hastighed.

2.1 Pickle

Pickle er Pythons indbyggede lagring mekanisme, som bruges til at gemme og genskabe komplette Python objekter direkte fra hukommelsen til en fil. Det betyder, at komplekse datastrukturer, klasser og objekter kan gemmes uden at skulle omskrives til simple talformater. I vores projekt anvendes Pickle til at gemme NEAT-genomer, fordi disse består af graf-baserede objekter med noder, forbindelser og metadata, som ikke kan repræsenteres som almindelige neurale netværks vægte. Pickle sikrer, at hele genomet kan genskabes præcist, så evolutions træningen kan fortsætte eller testes på nye baner. [6]

2.2 Keras

Keras er et high level deep learning framework bygget oven på TensorFlow og anvendes til at opbygge, træne og gemme neurale netværk. Det er designet til numeriske tensor baserede modeller, som trænes med gradient descent og backpropagation. I projektet bruges Keras til DDQNagentens neurale netværk, da denne agent arbejder med vægtmatricer og kontinuerlig

læring ud fra belønningssignaler. Keras gør det muligt at gemme hele modellen både arkitektur, vægte og optimizer tilstand, så agenten kan genindlæses og fortsætte træning eller evalueres på nye baner.[3] [2]

3 Analyse

3.1 GA-only: NEAT som styring via sensorinput og fitness

GA-only bygger på neuroevolution med NEAT, hvor både vægte og netværkets struktur kan udvikles over generationer (Stanley & Miikkulainen, 2002). Implementationen er baseret på `neat-python`, som understøtter centrale NEAT-mekanismer som speciation og elitisme [5].

Handlinger. Neuralt netværk modtager de fem normale afstande og returnerer to output. Output mappes til en diskret styring med punker. Hvis `out[0] > 0.7` drejer bilen i én retning, hvis `out[1] > 0.7` drejer den i den anden, og ellers kører den ligeud.

Progression og finish-logik. Fremskridtet måles via 10 checkpoints. Checkpoints er skrevet som referencekoordinater og skaleres til den aktuelle skærm hver frame (Resizeable). En gennemførsel registreres først, når alle checkpoints er passeret og bilen krydser finish. For at undgå for tidigt afslutning af banen er startpositionen "finish" først aktiv efter en kort arming-periode (1.5 sek).

Fitness og tidlig afslutning. Fitness beregnes løbende med en step-straf (der favoriserer hurtigere løsninger), bonus ved checkpoint-fremskridt og en større bonus ved gennemførsel, samt straf ved kollision. Derudover anvendes et stagnations kriterie, som afslutter biler, der ikke har gjort checkpoint-fremskridt i et givet antal steps, hvilket i vores tilfælde er 50. Det øger selektionen til nye generations der både er stabile og faktisk flytter sig frem på banen, og flere følger med.

Konfiguration og designvalg. Netværkene er feedforward og bruger `tanh` som aktivering. Strukturen kan ændres via mutationer, der tilføjer/fjerner forbindelser og noder. Populationen opdeles i species baseret på kompatibilitet, og elitisme sikrer, at de bedste genomer bevares på tværs af generationer, hvilket typisk forbedrer stabiliteten i træningen [5]

3.2 RL-only: DDQN som styring via sensorinput og reward shaping

I RL-only trænes en agent med Double Deep Q-Network (DDQN). Modellen estimerer $Q(s, a)$, dvs. hvor god en handling a er i en tilstand s målt som forventet samlet reward. I klassisk Q-learning kan man gemme disse værdier i en Q-tabel, men fordi vores observationer er kontinuerte altså flydende tal, bruger vi i stedet et neuralt netværk, der kan generalisere mellem lignende tilstande.

Tilstand (observationer). Agenten får 9 input: 8 raycasts, der måler hvor tæt bilen er på en væg i forskellige retninger omkring bilen, samt bilens hastighed normaliseret som $v/v_{\max}$. Afstandene skaleres til $[0, 1]$, så alle input har samme størrelse, og vi har valgt, at en højere værdi betyder, at bilen er tættere på en væg.

Handlinger Agenten vælger mellem 6 handlinger,(0) intet, (1) frem, (2) venstre, (3) højre, (4) frem+venstre, (5) frem+højre. DDQN-netværket outputter derfor 6 Q-værdier, og agenten vælger typisk den handling med højest Q-værdi.

Miljø og progression (gates) Banen er lavet med en ydre og en indre væg. Ud fra væggen genererer vi automatisk en række "gates" (goals), som agenten skal passere i rækkefølge. For hvert punkt laves en gate som en linje mod den nærmeste indre væg. Hver gang en bil har passeret en goal, aktiveres det næste.

Reward og afslutning af episode. Agenten får reward for at passere den aktive gate, for at komme tættere på gate-center (midten af banen), og for at pege i retning af næste gate. Derudover indgår hastighed som et mindre signal, og der gives straf, hvis agenten er meget tæt på væg, 0-1. En episode stopper ved kollision, hvis bilen kører ud af banen (out of bounds), ved timeout (maks. steps), eller hvis agenten i lang tid ikke laver fremskridt (ingen reward).

DDQN-træning. DDQN bruger to netværk, et online-netværk til at vælge handlingen og et target-netværk til at beregne target-værdien. Det mindsker overestimering og gør træningen mere stabil end standard DQN. Træning sker med replay buffer og minibatches, og exploration styres med ϵ-greedy, hvor ϵ gradvist falder til en minimumsværdi. [1]

3.3 Hybrid: NEAT-bias på DDQN's action-valg

I hybrid trænes en hel NEAT-population i et miljø med flere viler, mens DDQN samtidig lærer af transitions fra alle biler. Observationen er den samme som i RL-only (rays + speed), men fordi NEAT kun bruger 5 input, reducerer vi observationen til 5 udvalgte ray-features. NEAT giver to outputs, som vi oversætter til et simpelt styringshint (venstre/højre/ligefrem) med samme 0.7-tærskel som i GA.

DDQN vælger stadig den endelige handling blandt de 6 actions, men når den vælger greedy, giver vi et ekstra "skub" (bias) til de actions, der passer til NEAT's hint (fx venstre $\rightarrow$ {left, forward+left}). Styrken af dette skub styres af `NEAT_BIAS`. Fitness for hvert genome beregnes som summen af de rewards bilen får i sin episode. Biler der ikke laver fremskridt stoppes med

et stagnation-kriterie, og en episode kan også afsluttes efter et fast antal omgange (lap-baseret stop).

3.4 Læringsprincip: fitness (GA) vs. Q-værdier (RL)

GA og RL ændrer adfærd på to forskellige måder.

I GA-only (NEAT) evalueres hvert individ i populationen ved at køre en episode i miljøet og få en **fitness-score**. Fitness afhænger af det mål, vi har valgt at optimere (her: at gennemføre banen). Efter hver generation udvælges de bedste individer (selektion), og nye individer skabes via mutation (og evt. crossover afhængigt af NEAT). Læring sker derfor indirekte gennem evolution, hvor løsninger med høj fitness bliver mere sandsynlige i næste generation.

I RL-only (DDQN) optimeres adfærden ved at lære en **Q-funktion** $Q(s, a)$, som estimerer den forventede fremtidige belønning ved at vælge handling a i tilstand s. Klassisk RL kan beskrives med en Q-tabel, men i vores setup er tilstanden ikke opdelt i faste "bokse", fordi rays og hastighed er kontinuerte værdier (flydende tal). Derfor bruges et neuralt netværk som **funktion-approksimator** for Q-værdierne (Deep Q-Learning / DDQN). Agenten lærer ved at sammenligne det netværket forventede med det, der faktisk skete, og justerer vægtene, så estimatet bliver mere korrekt næste gang. Et ekstra *target*-netværk opdateres langsommere og bruges som en stabil reference, så træningen bliver mere stabil.

Kort sagt: **GA søger direkte efter en god policy gennem fitness-baseret selektion**, mens **RL lærer Q-værdier, som indirekte styrer policyen via greedy valg af handlinger**.

4 Resultater

4.1 Træning på Track 1: RL-only

Figur 1 viser træningsudviklingen for RL-only (DDQN) målt som hvor mange steps agenten overlever pr. episode og den samlede episode-score (reward). Kurven vises både som de rå værdier og som et glidende gennemsnit for at gøre udviklingen lettere at se.

Over træningen øges agentens stabilitet sig markant. I starten overlever den typisk kun 20–30 steps, mens de sidste 20 episoder ligger omkring 241 steps i gennemsnit. Den bedste episode når 912 steps, og rolling mean topper omkring 330. Episode-score følger samme mønster fra gennemsnitligt negativ i starten (ca. -4.5) til klart positiv senere (ca. 212), med en bedste episode omkring 875 og rolling mean omkring 290. Det indikerer, at agenten både overlever længere og i højere grad bevæger sig mod gates og opnår shaping-rewards.

Der ses dog store sving/dyk mellem episoder, hvilket er forventeligt pga. ϵ-greedy exploration og miljøets reward shaping. I dette run falder ϵ til minimum (0.10) omkring episode $\approx$ 92, så agenten stadig tager tilfældige handlinger i ca. 10% af tiden, hvilket bidrager

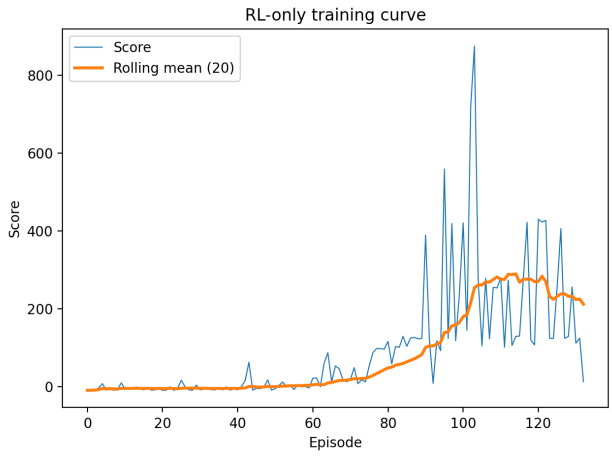

Figure 1: graf over mean

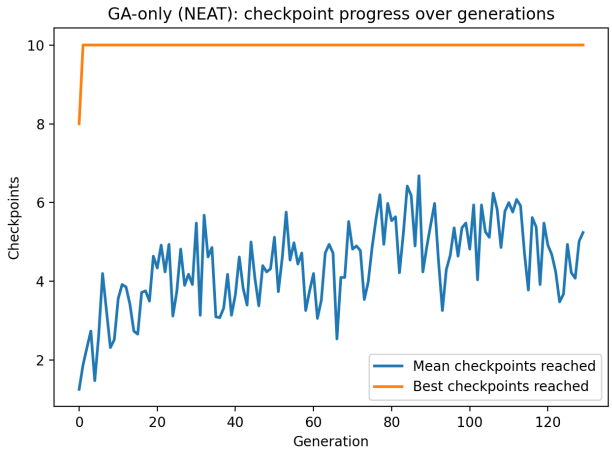

Figure 2: GA-only (NEAT): checkpoint-progress over generationer på træningsbanen.

til variationen. Samlet fungerer RL-only som en relevant baseline med tydelig læring, men ikke fuldt stabil adfærd.

4.2 Træning på Track 1: GA-only

I GA-only træningen på træningsbanen ses en tydelig forbedring i populationens gennemsnitlige progress over tid. I Figur 2 stiger *mean checkpoints reached* fra ca. 1 i de første generationer til omkring 5–6 senere i forløbet. Kurven er tydeligt støjende med mange sving/dyk, men den overordnede er opad, hvilket passer til en GA-træning, hvor mutation og selektion kan give perioder med både fremgang og midlertidige tilbagefald.

Samtidig ses, at *best checkpoints reached* ligger ved maksimum (10) næsten hele forløbet. Det betyder, at der tidligt opstår mindst én stærk løsning, som kan nå hele checkpoint-sekvensen, hvor én bil klare hele bane. Det centrale resultat er derfor ikke, at "der findes én god agent", men at gennemsnittet gradvist løftes, dvs. at en større del af populationen lærer at komme længere rundt på banen på baggrund af den ene bil, der klarede banen.

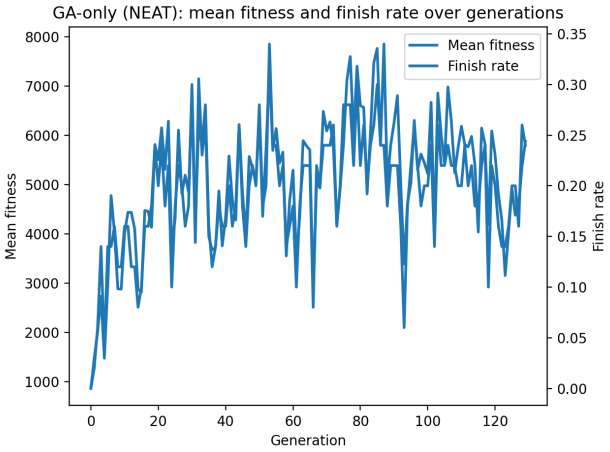

Figure 3: GA-only (NEAT): mean fitness og finish rate over generationer på træningsbanen.

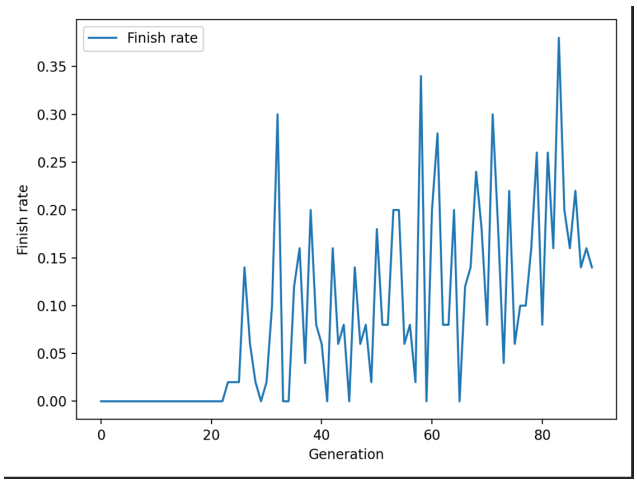

Figure 4: Hybrid: finish rate over generationer på træningsbanen.

Figur 3 viser udviklingen i mean fitness og finish rate. Mean fitness stiger kraftigt i begyndelsen og ligger derefter på et højere niveau, men med store udsving. Støjen er forventelig, fordi populationen konstant ændres af mutation/crossover, og fordi der i mange generationer stadig findes svage genomer, som trækker gennemsnittet ned. Finish rate varierer og ligger i perioder omkring 0.15–0.30 af bilerne gennemførte. Det viser, at flere end én enkelt agent kan klare banen, men at resultatet stadig svinger mellem generationer. Samlet viser graferne, at GA-only finder en fungerende strategi relativt tidligt (best-case), men at den langsommere forbedring handler om robusthed og udbredelse i populationen (mean checkpoints og finish rate).

4.3 Træning på Track 1: Hybrid (GA+RL)

Hybrid-modellen kombinerer NEAT og DDQN ved, at NEAT giver et simpelt styringshint (venstre/højre/ligefrem), mens DDQN vælger den endelige handling blandt de 6 diskrete actions. DDQN trænes samtidig under evalueringen via replay buffer, så policyen gradvist bliver bedre over generationer.

Figur 4 viser hybridens *finish rate*, altså hvor stor en del af populationen der gennemfører banen i hver generation. I de første generationer er finish rate 0, hvilket betyder, at ingen biler gennemfører. Efter omkring generation 20 begynder den at blive positiv, og i enkelte generationer når den op mod cirka 0.35. Det viser, at hybridmodellen går fra næsten ingen gennemførsel til perioder, hvor en tydelig del af populationen faktisk kan gennemføre banen.

Finish rate svinger dog mellem generationerne. Det giver mening i en hybridopsætning, fordi populationen hele tiden ændrer sig gennem evolution (NEAT), og fordi DDQN stadig udforsker med en lille sandsynlighed for tilfældige handlinger. Overordnet tyder udviklingen på, at gennemførsel bliver mere almindeligt senere i træningen, og at modellen kan lære en adfærd, som kan gennemføre banen på træningsbanen.

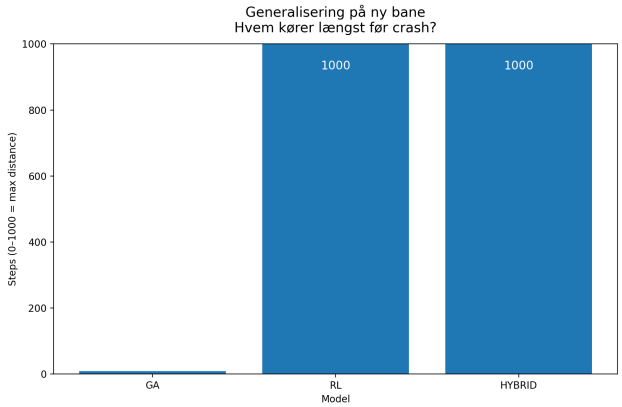

Figure 5: Generalisering på Track 2: antal steps før crash (1000 = ingen crash inden for grænsen).

4.4 Test på Track 2 (uset bane): Generalisering

Figur 5 sammenligner de tre modeller på Track 2 ved at måle, hvor mange steps de kan køre, før de crasher. Vi bruger et loft på 1000 steps, så 1000 betyder i praksis, at modellen ikke crashede inden for testens længde.

Resultatet viser en klar forskel: RL og hybrid når begge 1000 steps, hvilket tyder på, at de kan køre stabilt på den nye bane uden at ramme vægge i testperioden. GA-only klarer sig derimod markant dårligere og crasher næsten med det samme (meget få steps). Det tyder på, at GA-løsningen i dette setup ikke generaliserer lige så godt til Track 2 som RL og hybrid, mens DDQN-baserede modeller virker mere robuste på tværs af baner.

Når man også kigger på pygame, ser man også at selvom RL og hybrid starter på samme tid, ender hybriden med at være hurtigere og klare banen på bedere tid end RL, selvom alle modeller har samm inputs, for at gøre det fair.

4.4.1 GA-Only

GA-only agenten blev givet med det bedst præsterende genom efter 130 generationers træning på Track 1 via pickle. Ved test på Track 2 crashede GA-modellen meget hurtigt og nåede kun få steps, før den ramte væggen. Det tyder på, at den strategi der var effektiv på træningsbanen, ikke overførte godt til den nye banes geometri, og at GA-only i dette setup har begrænset generalisering.

4.4.2 RL-Only

RL-only agenten blev trænet i 130 episoder på træningsbanen. Under evalueringen på testbanen gennemførte agenten ligeledes banen succesfuldt. Selvom testbanen var anderledes end træningsbanen, så lykkedes det RL-only modellen at tilpasse sig det nye miljø og undgå kollisioner gennem hele kørslen.

4.4.3 Hybrid (GA + RL)

Hybrid-modellen blev trænet over 90 generationer, og resultaterne viste, at den præsterede bedst sammenlignet med de to øvrige modeller. Modellen formåede ikke blot at gennemføre testbanen, men udviste også en meget god generaliseringsevne på en ukendt bane, hvilket indikerer, at den ikke blot har memoriseret træningsbanen, men har lært mere generelle kørselsmønstre. Derudover viste hybrid-modellen sig at være hurtigere rundt om banen end RL-only modellen, hvilket tyder på, at kombinationen af genetisk optimering og reinforcement learning har givet en mere effektiv og stabil kørselsadfærd.

5 Konklusion

Formålet var at undersøge, om en kombination af GA (NEAT) og RL (DDQN) kan forbedre en agents evne til at køre uden kollisioner, og om en hybridmodel kan generalisere til en ny bane. På Track 1 viser både GA-only og RL-only tydelige forbedringer over tid. GA-only er populationens gennemsnitlige fremskridt over generationer bedere, mens RL-only lærer at overleve længere og opnå højere reward over episoder. Hybridmodellen opnår en positiv finish rate efter ca. generation 20 og viser, at gennemførsel bliver mere almindeligt senere i træningen.

I forhold til underspørgsmål 1 ses forskellige læringsmønstre. GA-only finder ret tidligt en enkelt bil, der kan klare banen, men det tager længere tid før mange af bilerne i populationen kan gøre det samme stabilt. RL-only bliver gradvist bedre, men der er stadig udsving, fordi agenten nogle gange vælger tilfældige handlinger og fordi belønningen er sammensat af flere dele. Hybridmodellen kombinerer det ved, at NEAT giver et simpelt styringshint, mens DDQN vælger den endelige handling, og det gør, at flere biler i nogle generationer kan gennemføre banen.

I forhold til underspørgsmål 2 viser testen på Track 2 en klar forskel. RL-only og hybrid når begge 1000 steps uden kollision inden for testgrænsen, mens GA-only crasher meget hurtigt. Det tyder på, at DDQN-baserede modeller i vores setup er mere robuste på tværs af baner end GA-only. Ved visuel inspektion i Pygame fremstår hybrid desuden hurtigere/mere effektiv end RL-only, men dette er ikke målt som en egentlig tid- eller hastigheds.

Samlet set viser resultaterne, at hybridmodellen klarer generalisering lige så godt som RL-only og ofte kører mere effektivt i praksis, mens GA-only i denne opsætning har sværere ved at overføre det den har lært til en ny bane.

Reference

[1] Hado van Hasselt, Arthur Guez, and David Silver. "Deep Reinforcement Learning with Double Q-Learning". In: *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence (AAAI-16)*. AAAI Press, 2016, pp. 2094–2100.

[2] Keras Team. *Serialization and saving (Keras guide)*. `https://keras.io/guides/serialization_and_saving/`. Accessed 2026-01-02.

[3] *Keras: The high-level API for TensorFlow*. `https://www.tensorflow.org/guide/keras/`. Accessed 2026-01-02.

[4] Yun-ting Liu et al. "Overview of Reinforcement Learning Based on Value and Policy". In: *2020 Chinese Control And Decision Conference (CCDC)*. IEEE, 2020, pp. 598–603.

[5] *NEAT-Python Documentation*. `https://neat-python.readthedocs.io/`. Accessed 2026-01-02.

[6] *pickle — Python object serialization*. `https://docs.python.org/3/library/pickle.html/`. Accessed 2026-01-02.